

F215 Digital Design — Tutorial 3

Subtractors, adder/subtractor design, scaling of adder circuits, and reading Verilog structurally

Question 1 — Building a Faster Subtractor

Two 1-bit values X and Y are to be subtracted ($X - Y$).

(a) Derive the half-subtractor truth table (D = difference, B = borrow) from first principles and find its minimal SOP via K-map — don't just recall it.

Solution: Truth table (D = difference, B = borrow):

X	Y	D	B
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Minimal SOP: $D = X'Y + XY' = X \oplus Y$ (checkerboard pattern on the K-map — no two adjacent cells agree, so this is already minimal).

$B = X'Y$ (a single minterm, already minimal).

(b) Prove algebraically (not just by citing the diagram) that a full subtractor is exactly two half-subtractors plus an OR gate: starting from $D_1 = X \oplus Y$, $B_1 = X'Y$, feed D_1 and B_{in} into a second half-subtractor to get D_2 , B_2 , and show D_2 and $(B_1 + B_2)$ match the standalone full-subtractor equations.

Solution: $D_2 = D_1 \oplus B_{in} = (X \oplus Y) \oplus B_{in} = X \oplus Y \oplus B_{in}$, matching the standalone full-subtractor's D exactly.

For the borrow:

$B_{out} = B_1 + B_2$

$$\begin{aligned} &= X'Y + (X \oplus Y)'B_{in} \\ &= X'Y + (XY + X'Y')B_{in} \\ &= X'Y + XYB_{in} + X'Y'B_{in} \\ &= X'(Y + Y'B_{in}) + XYB_{in} \quad [\text{absorption: } Y + Y'B_{in} = Y + B_{in}] \\ &= X'(Y + B_{in}) + XYB_{in} \\ &= X'Y + X'B_{in} + XYB_{in} \\ &= X'Y + B_{in}(X' + XY) \\ &= X'Y + B_{in}(X' + Y) \quad [\text{absorption: } X' + XY = X' + Y] \\ &= X'Y + X'B_{in} + YB_{in} \end{aligned}$$

— exactly the standard full-subtractor borrow equation.

(c) Just as carry-lookahead replaces rippling with a direct two-level circuit using “generate” G_i and “propagate” P_i signals, define analogous borrow-generate G_i and borrow-propagate P_i signals (as Boolean functions of X_i, Y_i) so that $B_{i+1} = G_i + P_i B_i$ matches your part (a) equation.

Solution: From $B_{out} = X'Y + X'B_{in} + YB_{in} = X'Y + (X' + Y)B_{in}$, matching the form $G_i + P_i B_i$ directly gives:

$G_i = X_i'Y_i$ (borrow-generate)

$P_i = X_i' + Y_i$ (borrow-propagate)

This P_i is OR-based, not XOR-based like the adder's propagate — trying $P_i = X_i \oplus Y_i$ here does not reproduce the correct borrow equation.

(d) Using your G_i, P_i , write the direct (non-recursive) expression for B_4 in terms of $G_0..G_3, P_0..P_3$, and B_{in} — mirroring the C_4 expression from lecture.

Solution: $B4 = G3 + P3G2 + P3P2G1 + P3P2P1G0 + P3P2P1P0 \cdot \text{Bin}$

Question 2 — A Life-Counter Bug

A hardware engineering team at a game studio is investigating a bug report about the circuit that updates a player's session life tally — net lives gained minus lives lost so far this session (which can legitimately be negative if a player has net-lost lives). The tally is computed by an S-controlled adder/subtractor circuit from lecture ($S=0$ adds, $S=1$ subtracts), but the bit-width used was never documented in the legacy code. Quality assurance has logged the following:

Tally before	Event	Tally shown after
1	Gained 1 life	2
2	Gained 2 lives	-4
3	Gained 1 life	-4
2	Lost 1 life	1
-3	Lost 3 lives	2
-4	Lost 3 lives	1

(a) Determine the bit-width being used here — the data only makes sense for one specific value. Then explain, in general terms, why certain rows give a wrong result and others don't (no need to trace bit-by-bit; describe the condition under which this circuit fails).

Solution: The data is only consistent with a 3-bit signed (2's-complement) representation, range $-4..+3$. Under this width, $2+2=4$ and $3+1=4$ both exceed the range, giving the wraparound results seen. General condition: overflow happens exactly when the two values actually being added inside the circuit (A and B when $S=0$; A and the 2's-complement of B when $S=1$) have the same sign, yet the computed result comes out with the opposite sign. Two operands of different sign can never overflow this way, since their true sum's magnitude is always smaller than the larger operand.

(b) Using only signals already present in the circuit (sum bits and internal carries — no extra adders), design the minimal additional logic for a flag that correctly indicates whenever this failure occurs. Verify against the table.

Solution: $V = C_{(n-1)} \oplus C_n$ — the XOR of the carry into the sign bit and the carry out of it (here, $V=C2 \oplus C3$). Checked against every row from the table above:

Tally before	Event	C2	C3	V
1	Gained 1 life	0	0	0
2	Gained 2 lives	1	0	1
3	Gained 1 life	1	0	1
2	Lost 1 life	1	1	0
-3	Lost 3 lives	0	1	1
-4	Lost 3 lives	0	1	1

The flag fires ($V=1$) on exactly the four rows QA flagged as buggy, and stays 0 on both rows marked “as expected.”

(c) Does your flag need separate handling for the subtract case ($S=1$, losing lives), or does it already work there unmodified? Justify using how the circuit internally handles subtraction, and check against the relevant rows in the table.

Solution: No modification needed. The circuit performs subtraction by internally adding A and B' with carry-in forced to 1, so from the adder's point of view every operation — $S=0$ or $S=1$ — is just an addition of two n-bit values. The same $V=C(n-1) \oplus C_n$ logic, sitting on the same two carry signals, automatically covers both modes, as already confirmed on the two $S=1$ rows in (b).

Question 3 — Reading Circuits, Not Writing Them

(a) Determine what this module computes, and simplify:

```
module mystery(input p, q, r, s, output f);
  wire w1, w2, w3;
  and (w3, w1, r);
  or  (f, w3, w2);
  and (w1, p, q);
  or  (w2, q, s);
endmodule
```

Solution:

$$\begin{aligned} f &= w3 + w2 \\ &= (w1 \cdot r) + (q + s) \\ &= (pq)r + q + s \\ &= pqr + q + s \\ &= q + s \quad \text{[absorption: } q + pqr = q, \text{ regardless of } p, r] \end{aligned}$$

The p and r inputs have no effect on the output at all.

(b) A colleague builds a full subtractor from two half_subtractor instances (ports .x, .y, .d, .b) like this:

```
module full_subtractor(input x, y, bin, output d, bout);
  wire d1, b1, b2;
  half_subtractor HS1(.x(x), .y(y), .d(d1), .b(b1));
  half_subtractor HS2(.x(bin), .y(d1), .d(d), .b(b2));
  or (bout, b1, b2);
endmodule
```

Draw the circuit this code describes, labeling every wire. Then determine: is d correct for every input combination? Is bout? If not, find a specific (x, y, bin) where it fails, and identify exactly which port connection is wrong.

Solution:

d is correct for every input, since $D = X \oplus Y$ is symmetric — swapping which signal is 'x' and which is 'y' doesn't change the difference bit.

bout is not: HS2 is instantiated as .x(bin), .y(d1), so its internal borrow output is $\text{bin}' \cdot d1$, but the correct second-stage borrow (from Question 1(b)) needs $d1' \cdot \text{bin}$ — a half-subtractor's borrow output is not symmetric in its inputs.

Concretely, for $(x, y, \text{bin}) = (0, 0, 1)$: $d1=0$, $b1=0$.

Correct: $b2 = d1' \cdot \text{bin} = 1 \cdot 1 = 1$, giving $\text{bout}=1$ (matches a true borrow, since $0-0-1=-1$).

As wired: $b2 = \text{bin}' \cdot d1 = 0 \cdot 0 = 0$, giving $\text{bout}=0$ — wrong.

Fix: HS2 should be .x(d1), .y(bin), not .x(bin), .y(d1).

(c) A testbench for full_subtractor is written as:

```

module tb_full_subtractor;
  wire t_x, t_y, t_bin;
  reg  t_d, t_bout;

  full_subtractor DUT(.x(t_x), .y(t_y), .bin(t_bin), .d(t_d), .bout(t_bout));

  initial begin
    t_x = 0; t_y = 0; t_bin = 0;
    #5 t_x = 0; t_y = 1; t_bin = 0;
  end
endmodule

```

Will this simulate correctly? If not, identify every incorrect declaration and explain — from what reg and wire actually represent, not “because that’s the rule” — why each one is wrong.

Solution:

No — this will not simulate correctly.

`t_x`, `t_y`, and `t_bin` are declared wire, but are assigned directly inside the initial block (`t_x=0; ...`); a wire can only be driven by a continuous source (a gate output or an assign statement), never by a direct procedural assignment — these should be reg.

Conversely, `t_d` and `t_bout` are declared reg, but are only ever connected to the DUT’s output ports, which drive them continuously — nothing in the testbench itself procedurally assigns to them, so they should be wire, not reg.

Every one of the five declarations has the wrong type.

Question 4 — How Do Adders Scale?

(a) For an n -bit ripple-carry adder (n full adders, each a fixed small number of gates; worst-case delay $2n+1$ gate-levels, from lecture), express total gate count and delay as functions of n . State how each grows as n increases.

This kind of growth-rate description is usually written using Big-O notation: $O(n)$ means “grows roughly proportionally to n ,” $O(n^2)$ means “roughly proportional to n^2 ,” $O(\log n)$ means “grows much more slowly — roughly the number of times you can repeatedly divide n by some fixed factor before reaching 1,” and $O(1)$ means “stays roughly constant regardless of n .”

Solution: Gate count grows as $O(n)$ — n full adders, each built from the same fixed handful of gates, so total gate count is directly proportional to n . Delay also grows as $O(n)$ — the $2n+1$ formula is exactly linear in n .

(b) Now take the same generate/propagate circuit from lecture and imagine building it for a general n -bit adder, the same way, just bigger. How do its total gate count and its delay scale with n ? (Trying it out for a few small values of n first may help before generalizing.)

Solution: Delay stays $O(1)$ — a small constant number of gate-levels (P/G computation, then the two-level AND-OR carry logic, then the final sum XOR), regardless of how large n gets, since every carry is produced by a flat two-level circuit. Gate count, however, grows as $O(n^2)$: C_i needs $i+1$ product terms, and summing $(i+1)$ for $i=1$ to n gives roughly $n^2/2$ — quadratic growth. The practical problem is gate fan-in: the AND gate for the last product term of C_n needs $n+1$ inputs, which isn’t physically realizable for large n .

(c) In practice, adders are built by taking 4-bit CLA units (as in lecture) and combining them using the carry-out of one feeding the carry-in of the next — the same way full adders were chained into a

ripple-carry adder. For a 64-bit adder built this way (16 such 4-bit blocks), express the total worst-case delay in terms of the number of blocks and whatever per-block delay you found relevant in (b). How does this scale with n ? Compare it to both (a) and (b) — what has this approach actually improved, and what hasn't it changed?

Solution: Total delay = (number of blocks) $\times$ (per-block constant delay from (b)) = $(n/4) \times \text{constant}$, which is still $O(n)$ — linear in n , just with a much smaller constant factor than pure ripple-carry (roughly a 4 $\times$ reduction in slope). What's improved is the constant factor, not the growth rate: doubling n still roughly doubles the delay, exactly as it did for plain ripple-carry.

(d) Here's a way to actually change the growth rate. Apply the same generate/propagate trick to the blocks themselves: define a block-generate signal G_{blk} (this block produces a carry regardless of its incoming carry) and a block-propagate signal P_{blk} (an incoming carry sails straight through the whole block), each as a Boolean function of that block's own 4 bit-level G_i, P_i . A second-level lookahead unit — structurally identical to the one from (b), just one level up — can then compute every block's incoming carry directly, instead of rippling block to block. Applied recursively: argue that overall delay becomes $O(\log n)$ while total gate count stays $O(n)$ (a structural argument is enough — no rigorous proof needed). For $n=64$ with 4-bit blocks: how many levels of hierarchy are needed? Roughly how many gate-delays does this give you, compared to the 129 from lecture?

Solution: Recursion depth = $\log_4(64) = 3$ levels of hierarchy. Using the same small constant per-level delay from (b)/(c) (roughly 3–4 gate-levels per level), total delay is only around $3 \times 4 \approx 12$ gate-delays — compared to 129 for the pure ripple-carry adder, roughly a 10 $\times$ improvement.

Because each level's hardware is proportional to the number of groups at that level (64, then 16, then 4, then 1), which shrinks geometrically, the total gate count across all levels sums to $O(n)$, not $O(n^2)$ — unlike the single flat CLA from (b).